

数据结构入门

陈仕杰

上海大学

2025 年 1 月



① 简介

② 基本数据结构及其 STL 实现

栈

队列

堆

③ 并查集

④ ST 表

⑤ 树状数组

⑥ 总结

栈

栈是一种后进先出 (last in first out) 的数据结构。

洛谷 B3614 【模板】栈

请你实现一个栈 (stack), 支持如下操作:

- `push(x)`: 向栈中加入一个数 x 。
- `pop()`: 将栈顶弹出。如果此时栈为空则不进行弹出操作, 输出 Empty。
- `query()`: 输出栈顶元素, 如果此时栈为空则输出 Anguei!。
- `size()`: 输出此时栈内元素个数。

栈的 STL 实现

```
1  int n; cin >> n;
2  stack<unsigned long long> s;
3  for (int i = 1; i <= n; ++i) {
4      string op; cin >> op;
5      if (op == "push") {
6          unsigned long long x; cin >> x;
7          s.push(x);
8      } else if (op == "pop") {
9          if (s.empty()) { cout << "Empty\n"; }
10         else { s.pop(); }
11     } else if (op == "query") {
12         if (s.empty()) { cout << "Anguei!\n"; }
13         else { cout << s.top() << "\n"; }
14     } else { cout << s.size() << "\n"; }
15 }
```

栈

洛谷 P1739 表达式括号匹配

检查一个括号表达式是否匹配。

将左括号视作进栈，将右括号视作出栈，栈空即为配对成功。

单调栈

通过保证栈内元素单调来求解问题的一类算法，叫做单调栈。

洛谷 P5788 【模板】单调栈

第 i 个元素之后第一个大于 a_i 的元素的下标

单调栈

构造一个从底到顶单调递减的单调栈。

从左到右遍历 a_i ，强制让其进栈。

为保证栈的单调性， a_i 进栈前先弹出若干个元素 $b_1, b_2, \dots, b_t$ 。

只有 b_j 右侧第一个元素会使它弹出。此时就找到了 $b_1, b_2, \dots, b_t$ 右侧的第一个元素，即 a_i ，对应修改标记即可。

每个元素只会进栈一次、出栈一次，均摊复杂度 $O(n)$ 。

单调栈

```
1  cin >> n;
2  stack<int> s;
3  for (int i = 1; i <= n; ++i) {
4      cin >> a[i];
5      while (!s.empty() && a[s.top()] < a[i]) {
6          f[s.top()] = i; s.pop();
7      }
8      s.push(i);
9  }
10 for (int i = 1; i <= n; ++i) {
11     cout << f[i] << " ";
12 }
```

注意代码中使用了 & 运算的短路效应，先判断栈是否为空，如为空就不继续操作，避免了可能的越界情况。

① 简介

② 基本数据结构及其 STL 实现

栈

队列

堆

③ 并查集

④ ST 表

⑤ 树状数组

⑥ 总结

队列

队列是一种先进先出 (first in first out) 的数据结构

洛谷 B3616【模板】队列

请你实现一个队列 (queue)，支持如下操作：

- `push(x)`：向队列中加入一个数 x 。
- `pop()`：将队首弹出。如果此时队列为空，则不进行弹出操作，并输出 `ERR_CANNOT_POP`。
- `query()`：输出队首元素。如果此时队列为空，则输出 `ERR_CANNOT_QUERY`。
- `size()`：输出此时队列内元素个数。

队列

```
1  int n; cin >> n; queue <int> q;
2  for (int i = 1; i <= n; ++i) {
3      int op; cin >> op;
4      if (op == 1) { int x; cin >> x; q.push(x); }
5      else if (op == 2) {
6          if (q.empty()){cout<<"ERR_CANNOT_POP\n"
7              ;}
8          else { q.pop(); }
9      } else if (op == 3) {
10         if (q.empty()){cout<<"ERR_CANNOT_QUERY\n"
11             ";"}
12         else { cout << q.front() << "\n"; }
13     } else {
14         cout << q.size() << "\n";
15     }
16 }
```


单调队列/滑动窗口

通过保证队列内元素单调来求解问题的算法，叫做单调（双端）队列。

洛谷 P1886 滑动窗口/【模板】单调队列

有一个长为 n 的序列 a ，以及一个大小为 k 的窗口，维护窗口移动过程中的最大值和最小值。例如，对于序列 $[1, 3, -1, -3, 5, 3, 6, 7]$ 以及 $k = 3$ ，有如下过程：

窗口位置								最小值	最大值
[1	3	-1]	-3	5	3	6	7	-1	3
1	[3	-1	-3]	5	3	6	7	-3	3
1	3	[-1	-3	5]	3	6	7	-3	5
1	3	-1	[-3	5	3]	6	7	-3	5
1	3	-1	-3	[5	3	6]	7	3	6
1	3	-1	-3	5	[3	6	7]	3	7



单调队列

此处仅考虑最大值的做法。

构造一个从队首到队尾单调递减的单调队列。

从左到右遍历，如果此时队首已经不在窗口内，则出队。

记此时遍历到 a_i ，若 a_i 大于队尾 a_j ，先弹出队尾若干个元素 $b_1, b_2, \dots, b_t$ ，再进队。

此时队列中从队首到队尾 a 值递减，下标 i 递增。每个元素进队一次，出队一次，均摊复杂度 $O(n)$ 。

单调队列

Fun Fact

将 a 视作实力，将 i 视作入学年份。

不难发现单调队列揭示了 ACM 届一个残酷的事实：

- 如果集训队里有学长学姐比你强，你可以把他们熬走
- 但如果有人比你年轻还比你强，那你就可以自觉退役了

单调队列

```
1 deque<int> qmax;
2 for (int i = 1; i <= n; ++i) {
3     while (
4         !qmax.empty() && qmax.front() <= i-k)
5     {
6         qmax.pop_front();
7     }
8     while (
9         !qmax.empty() && a[i] > a[qmax.back()])
10    {
11        qmax.pop_back();
12    }
13    qmax.push_back(i);
14    if (i >= k) {
15        cout << a[qmax.front()] << " ";
16    }
17 }
```

① 简介

② 基本数据结构及其 STL 实现

栈

队列

堆

③ 并查集

④ ST 表

⑤ 树状数组

⑥ 总结

堆

堆是一种可以在 $O(\log n)$ 的时间内维护一个最值的数据结构。

维护最大值的称为大根堆，维护最小值的称为小根堆，堆顶存放着最值。

堆可以用 STL 中的优先队列实现。

堆的应用只有一个，就是求最值，但求什么最值、求出最值后怎么用，需要根据题目具体分析。

优先队列的使用

存放已有的数据类型时，直接声明 `priority_queue<>`

- 方式一：只写一个数据类型，如 `priority_queue<int> q`，默认为大根堆。

- 方式二：三件套。

`priority_queue<数据类型, vector<数据类型>, 比较方式>`。

- 比较方式有两种，`greater<数据类型>` 和 `less<数据类型>`，分别表示当数字大/小时该数字下沉，也就对应了小根堆和大根堆。
- 需要特别注意：`greater` 是小根堆！`greater` 是小根堆！大于表示大数字下沉而小数字在堆顶！

优先队列的使用

存放自定义的结构体时，需要重载小于号，以供 `priority_queue` 调用。重载的小于号中，实现小于功能为大根堆，实现大于功能为小根堆。

```
1 struct node {  
2     // 这里写一些结构体中的变量  
3     int x;  
4     // 这里是重载部分  
5     const bool <(const node& x) const  
6     {  
7         return x < b.x; // 大根堆  
8         // return x > b.x; 就是小根堆  
9     }  
10 };  
11 priority_queue<node> q;
```


洛谷 P1090 [NOIP2004 提高组] 合并果子

n 堆果子，每次可以选择两堆合并，消耗的体力等于两堆果子的重量之和，求 $n - 1$ 次合并操作最少消耗多少体力。

堆

考虑贪心。

进行 k 次操作后，剩余 $n - k$ 堆果子，最优的方案是下一次选择这 $n - k$ 堆中重量最小的两堆合并。

```
1  int n; ll ans=0;
2  priority_queue<ll, vector<ll>, greater<ll>> q;
3  for (int i = 1; i <= n; ++i) {
4      int a; cin >> a; q.push(a);
5  }
6  for (int i = 1; i < n; ++i) {
7      ll x1 = q.top(); q.pop();
8      ll x2 = q.top(); q.pop();
9      ans += x1 + x2;
10     q.push(x1 + x2);
11 }
12 cout << ans << "\n";
```

对顶堆

P7072 [CSP-J2020] 直播获奖

前 p 个人，获奖名额有 $\max(1, \lfloor p \times w\% \rfloor)$ 个，如有选手成绩相同则都能获奖。对于 $1 \leq p \leq n$ ，实时更新获奖分数线。

对顶堆

构造一对大根堆和小根堆，小根堆存大数，大根堆存小数。

保证小根堆的数量为获奖人数，少了就从大根堆里加数字，多了就把数字丢到大根堆里。

更新完成后，小根堆堆顶的数字即为分数线。

对顶堆

```
1 priority_queue<int> q;  
2 priority_queue<int, vector<int>, greater<int>> p;  
3 for (int i = 1; i <= n; ++i) {  
4     int x, k=max(1, i*w/100);  
5     cin >> x;  
6     if (q.empty() || x > q.top()) { p.push(x); }  
7     else { q.push(x); }  
8     while (!p.empty() && p.size() > k) {  
9         q.push(p.top()); p.pop(); }  
10    while (!q.empty() && p.size() < k) {  
11        p.push(q.top()); q.pop(); }  
12    cout << p.top() << " ";  
13 }
```

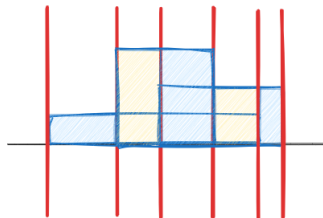
堆的删除——懒删除

P2061 [USACO07OPEN] City Horizon S

给定 n 个横平竖直的矩形，保证它们下底边与 x 轴重合，求矩形面积并。

堆的删除——懒删除

从左往右，在有建筑开始或结束的地方，放一条假想的、垂直于地平线的【扫描线】，对合并完之后的那个奇形怪状的图形做切割。



这样一来，每次切割出来的部分是一个矩形。面积可以拿相邻两条扫描线的距离差 Δx ，乘上这段里面的高度最大值 h_{max} 。答案是这 $2n$ 条扫描线切割出来的面积之和，即 $S = \sum(\Delta x \times h_{max})$ 。

堆的删除——懒删除

因为扫描线是人为定下来的， Δx 很好求，难点在于如何维护 h_{max} 。

假设我们已经维护出了前 i 条扫描线 bd_i ，对于第 $i + 1$ 条扫描线 bd_{i+1} ：

- 两条扫描线之间的距离差为 $\Delta x = bd_{i+1}.x - bd_i.x$ ，从堆中取出 h_{max} ，将 $\Delta x \cdot h_{max}$ 计入答案。
- 如果这条扫描线标记了某一幢建筑的【起点】，那么将这幢建筑的高度加入堆中。
- 如果这条扫描线标记了某一幢建筑的【终点】，那么执行删除操作。

堆的删除——懒删除

考虑如何从堆中删除一个数字。

因为在堆中，我们只能很快知道堆顶的元素，对于具体元素的位置需要遍历整个堆，时间吃不住。

但是，我们可以发现，如果被删去的元素不在堆顶，那么将不会从堆中取出被删去的元素，进而也就不会对答案产生影响。

所以我们可以用 `flag` 数组，或者 `cnt` 数组，或者再开一个堆，记下所有要删除的元素。

每次从堆中取出元素时，检查它是否被删除，然后才真正执行删除操作。

堆的删除——懒删除

```
1  #include <bits/stdc++.h>
2  #define ll long long
3
4  using namespace std;
5
6  const int MAXN=4e4+5;
7  int n;
8  ll ans;
9
10 struct node {
11     ll x, h;
12     bool st;
13 } bd[MAXN<<1];
14
15 priority_queue <ll> q, del_q;
```

堆的删除——懒删除

```
1  int main()
2  {
3      cin >> n;
4      for (int i = 1; i <= n; ++i) {
5          long long a, b, h; cin >> a >> b >> h;
6          bd[2*i-1] = { a, h, 1 };
7          bd[2*i]   = { b, h, 0 };
8      }
9      sort(bd+1, bd+2*n+1, [](node a, node b){
10         return a.x<b.x;});
11     for (int i = 1; i <= 2*n; ++i) {
12         while (
13             (!q.empty()) &&
14             (!del_q.empty()) &&
15             (q.top() == del_q.top())) {
16             q.pop(); del_q.pop();
17         }
18     }
```

堆的删除——懒删除

```
1      }
2      if (!q.empty()) {
3          ans += q.top()*(bd[i].x-bd[i-1].x);
4      }
5      if (bd[i].st) { q.push(bd[i].h); }
6      else          { del_q.push(bd[i].h); }
7  }
8  cout << ans << endl;
9  }
```

- 1 简介
- 2 基本数据结构及其 STL 实现
- 3 并查集**
- 4 ST 表
- 5 树状数组
- 6 总结

并查集

并查集是一种维护动态连通性的数据结构。

洛谷 P3367 【模板】并查集

维护两种操作：合并两个集合；判断两个数是否在同一个集合内。

每个连通块选取一个组长，每个点的组长记为 f_i ，初始时 $f_i = i$ 。
记 $\text{find}(x)$ 表示查询 x 的最高级组长（满足 $f_i = i$ 的组长）。

- $\text{merge}(x, y)$ ，可以视为将 $f[\text{find}(x)]$ 更新为 $\text{find}(y)$ 。
- $\text{same}(x, y)$ ，可以视为查询 $\text{find}(x)$ 与 $\text{find}(y)$ 是否相等。

并查集在最小生成树中有应用（上节课已讲）。

```

1 struct dsu {
2     int n;
3     vector <int> fa;
4     vector <ll> cnt;
5     Dsu(int n): n(n) {
6         fa.assign(n+1, 0);
7         iota(fa.begin(), fa.end(), 0);
8         cnt.assign(n+1, 1);
9     }
10    int find(int x) {
11        return fa[x]==x? x: fa[x]=find(fa[x]);
12    }

```

```
1 void merge(int x, int y) {
2     x = find(x);
3     y = find(y);
4     fa[y] = x;
5     cnt[x] += cnt[y];
6 }
7 void add(int x) {
8     ++cnt[find(x)];
9 }
10 };
11 dsu d(n);
```


ST 表

对每个位置 j , 我们需要预处理区间 $[j, j + 2^i - 1]$ 的最大值, 其中 $1 \leq i \leq \lfloor \log_2 n \rfloor$ 。

不妨将其简记为 $f(i, j)$ ，不难看出递推式：

$$f(i, j) = \max(f(i-1, j), f(i-1, j+2^{i-1}))$$

共 $O(n \log n)$ 个状态, $O(1)$ 转移, 预处理时间复杂度 $O(n \log n)$ 。

取 $s = \lceil \log_2(r-l+1) \rceil$, $\text{query}(l,r)=\max(f(l,s),f(r-2^s+1,s))$;

单次询问时间复杂度 $O(1)$ 。

44 / 55

```

1  template <typename T>
2  struct ST {
3      int n=0, I=0;
4      vector<int> Log;
5      vector<vector<T>> st;
6      ST() { }
7      ST(const vector<T>& a) {
8          n = a.size();
9          Log.assign(n+1, 0);
10         for (int i = 2; i <= n; ++i) {
11             I=Log[i]=Log[i/2]+1;
12         }
13         st.assign(I+1, vector<T>(n));
14         copy(a.begin(), a.end(), st[0].begin());

```

45 / 55

◀ ◻ ▶ ◀ ◼ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻

树状数组

P3374 【模板】树状数组 1

已知一个数列，维护两种操作：将某一个数加上 x ；求出某区间每一个数的和

如果没有修改操作，你会怎么处理？

如果只有一次询问操作，你会怎么处理？

48 / 55

49 / 55

知乎 @Zqlceberg

50 / 55

000

○○

六二米
〇〇〇〇

○○○○○

内訳表

000

000

00000000000000000000000000000000000000

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

- <https://www.cnblogs.com/LittleDrinks/p/17737926.html>
- <https://www.cnblogs.com/LittleDrinks/p/17763322.html>